CLAUDE

THE COMPLETE GUIDE

Cowork · Skills & Plugins · Claude Code · Claude Chat

COWORK SKILLS PLUGINS CODE

by **Krystian Wojtarowicz & Damian Danelczyk**

WHAT'S INSIDE

Enjoying the Course?

Your feedback helps us improve and helps other learners discover this course.

WHY IT MATTERS

- ❖ Helps us improve the course content
- ❖ Guides other learners to find it
- ❖ Takes less than 60 seconds

HOW TO LEAVE A REVIEW

1. Open the course page on Udemy
2. Watch a few videos to unlock the rating
3. Click "Leave a Rating" — top right corner

Thank you from Krystian & Damian ❤

CLAUDE COWORK

Most desk work doesn't need a human.

It needs an AI that understands context, takes action, and works in plain language.

THE OLD WAY

Hours lost to repetitive work.

You're the glue between your apps — and it eats your day.

- Organize, rename and move files manually
- Format the same reports week after week
- Switch between apps, losing focus each time

~2.5 hrs/day lost to avoidable tasks

CLAUDE COWORK

Your AI desktop co-worker.

Describe any task. Cowork manages your files, writes reports, does research, and automates workflows — right on your computer.

- Lives on your actual desktop
- No code, no technical setup
- Files, docs, research, apps — all of it

Plain language → instant action

WITH COWORK

Done. While you focus.

You describe the outcome. Cowork takes care of the rest.

- Files sorted and organized automatically
- Reports generated on demand
- Research done in seconds, not hours

Reclaim your day — every single day

Files & Folders | Documents & Reports | Web Research | App Automation | Data & Analysis

No code. No new tools. No technical setup. **Just describe what you need** — and Cowork gets it done.

Available in the Claude desktop app — Research Preview

Typical LLMs vs Cowork

LLMs give you the output inside a chat. Cowork delivers the result into your actual environment.

TASK	TYPICAL LLM Advises only — you still do the work	CLAUDE + COWORK Actually executes — task is done
Write a document	GIVES YOU: Outputs the content as text — basic file export with some tools *Result stays in the chat; formatting and filing are still on you*	ACTUALLY DOES: DONE Creates a formatted .docx file in your chosen folder *File ready to open, share, or edit immediately*
Organize files	GIVES YOU: Explains the steps you should take to sort your files *You still navigate folders, move files, do the renaming*	ACTUALLY DOES: DONE Reads your folder, moves, renames, and organizes files *Folder organized — zero manual steps required*
Research a topic	GIVES YOU: Most can search the web and return a text summary in chat *Answer lives in the chat — you still compile, save, and organize it*	ACTUALLY DOES: DONE Searches the web in real time, compiles a saved report *Fresh research delivered as a readable document*
Make a presentation	GIVES YOU: Outputs slide content as text — tools may export a basic file *Content stays in the chat; designing and building the file is on you*	ACTUALLY DOES: DONE Builds and delivers a fully designed, ready .pptx file *Open it, present it — done*
Send a follow-up email	GIVES YOU: Drafts the email text for you to copy-paste and send *You still switch to your email client and send it yourself*	ACTUALLY DOES: DONE Drafts and sends the email via Gmail — after your approval *Email sent — no app-switching, no extra steps*

THE SHIFT

Every tool before Cowork was a **smart advisor** — it told you exactly what to do, then waited for you to do it. Cowork is the first time Claude can **close the loop** — plan, execute, and deliver without switching apps.

ORIGIN STORY

Why Cowork Was Built

It started with engineers doing strange things in the terminal — and everyone else watching from the outside.

ACT 01

Developers Got Superpowers

Claude Code launched

Engineers could finally delegate coding from the terminal — write, run, debug, and fix without switching contexts.

Then something unexpected happened

Developers started using it for everything. Not just code — for doing things with folders, documents, emails, automations, or even... planning vacations

It became an AI operator

"Organize this folder." "Draft my standup update." "Summarize these logs and email the team."

> *"I haven't manually moved a file in weeks."*
> *— engineer using Claude Code*

ACT 02

Everyone Else Was Locked Out

Non-devs saw what was happening

Teammates, managers, and coordinators noticed. "Can you set that up for me?" became a daily request.

The wall was immediate

Claude Code requires a terminal, bash commands, git, and code knowledge. Most professionals hit a dead end in under 30 seconds.

The gap was uncomfortable

The most powerful AI tool in the building was locked behind a $ prompt that most people had never seen before.

> *"I just want it to DO the thing.*
> *Why do I need to learn bash?"*
> *— literally everyone else*

ACT 03

Cowork Was the Answer

One question changed everything

What if the same execution power existed — with no terminal, no commands, no code?

Same Claude. Different front door.

Cowork is the desktop interface built for professionals who work with files and apps, not codebases.

The problem solvers finally had a tool

Marketing managers. Course creators. Consultants. Business owners. The people who needed it most — finally included.

> *"Finally. It just does it."*
> *— Cowork user, first session*

THE REAL REASON

Cowork exists because **the desire to have AI execute tasks isn't a developer trait** — it's a human one. The only thing that was missing was a tool built for **the rest of us.**

Cowork is not a dumbed-down Claude Code — it is the same execution power, built for the people who needed it first

Who Is Cowork For?

The professional who wants AI to do the work — not explain how to do it.

IDEAL COWORK USER

The Professional Non-Developer

THIS IS YOU IF...

- You work in Word, Excel, or PowerPoint — not code editors
- You want Claude to complete the task — not give you steps to follow
- Your day involves repetitive file, doc, or email work
- You've tried ChatGPT but still had to do all the real work yourself
- You want AI that works across your apps, not just in a chat box

THEIR DAILY TOOLS: Word | Excel | Gmail | PowerPoint | Notion

FITS PERFECTLY FOR

- Course Creators
- Marketing Managers
- Business Owners
- Consultants
- Content Creators
- Educators & Trainers
- Executive Assistants
- Solopreneurs

NOT THE RIGHT FIT FOR

- x Developers building with code → *Claude Code*
- x API builders and engineers → *Anthropic API*
- x Casual chatters who need no files → *Claude Chat (free)*

THE TEST

Ask yourself: **"Do I want to be told how, or do I want it done?"** If you want it **done** — you're in the right course.

CLAUDE CODE VS CLAUDE COWORK

Same Claude engine. Two entry points.

One is built for developers. One is built for everyone else.

FOR DEVELOPERS

Claude Code

A coding agent in your terminal. Builds, runs, and deploys software with full codebase and system access.

- Executes code, builds and deploys software
- Full codebase, git, and terminal access
- Requires developer & terminal knowledge
- Most non-developers never try it

Powerful — but developer-only

VS

FOR EVERYONE

Claude Cowork

Your AI desktop co-worker. Manages files, writes reports, does research, and automates workflows — in plain English.

- Lives on your actual desktop
- Plain English — no commands needed
- Zero technical knowledge required
- Available to anyone with Claude

Plain language → instant action

Claude Code unlocks Claude for developers. **Cowork** unlocks Claude for **everyone** — no terminal, no code, no setup.

Both are built on the same Claude AI. The only difference is who gets to use them — and how.

EXERCISE

Organize Your Files with Cowork

Pick a real messy folder — rename files, sort into category folders, remove duplicates — then share the result

1 Pick Your Messy Folder

STEP 1 OF 3

- Find a folder on your computer with files that need work — invoices, downloads, anything messy
- Bonus: use something with mixed dates, inconsistent names, or obvious duplicates
- The messier the better — this is exactly where Cowork earns its keep

2 Give Cowork the Instructions

STEP 2 OF 3

- Open Cowork, select your folder, then tell Claude what to do in plain English
- Try: rename files by date, sort into category folders by type, remove any duplicates
- Watch it execute in real time — no code, no commands, just a clean result

3 Share Your Before & After

STEP 3 OF 3

- Take a screenshot of the folder before you start AND after Cowork is done
- Post both screenshots in our Skool community exercise thread (link in video resources)
- See how other students tackled their folders — every messy-to-clean transformation counts!

HOW TO SHARE YOUR RESULT

- Post a Screenshot
- Share Before & After
- Describe What You Organized

EXERCISE

Use Claude's Browser Agent

Pick a real task you'd normally do online — let Claude navigate and act — then share what it did

1 Pick a Browser Task

STEP 1 OF 3

- Any task you'd normally do in a browser — search, research, create a page, post, fill a form
- Ideas: search X for AI news, open Notion and create a page, draft and publish a LinkedIn post
- The more real and useful the task, the more satisfying the result

2 Let Claude Navigate

STEP 2 OF 3

- Open the browser agent, describe your task in plain English, then watch Claude execute
- Guide it with follow-up messages if it goes off track — it responds like a teammate
- Try a vague brief first, then refine — see how well Claude interprets your intent

3 Share What Happened

STEP 3 OF 3

- Screenshot or screen record the moment Claude does something that surprises you
- Post in Skool: what you asked, what Claude actually did, and what you'd try next
- See the most creative use cases from classmates — browser tasks have no limits

HOW TO SHARE YOUR RESULT

- Post a Screenshot
- Share What Claude Did
- Describe Your Use Case

Research & Analysis

Use Cowork to gather, analyze, and organize information at scale

COMPETITIVE INTEL

15+ Competitor Sites at Once

Give Claude a list of websites — it visits each one, extracts pricing, features, and target audience, then compiles everything into a clean comparison table

KNOWLEDGE SYNTHESIS

5 Articles, One Summary Doc

Feed Claude a reading list — it extracts key insights, spots common themes across sources, and builds an organized summary with source citations

INSTANT NOTES

Any Webpage to Notes or Flashcards

Reading something useful? Ask Claude to transform the current page into structured notes or study flashcards instantly — no copy-paste required

FILE & MEDIA

Rename 50+ Photos in Bulk

Point Claude at a folder of images — it renames every file based on what it contains and automatically moves videos into their own subfolder

THE REALITY These aren't demos — they're real tasks Claude Cowork handles in minutes, not hours

Inbox & Admin

Let Cowork handle the repetitive communication and admin work you keep putting off

EMAIL FOLLOW-UPS

Find Emails That Got No Reply

Claude scans your sent mail from the past 30 days, flags messages with no response, and drafts a contextually appropriate follow-up for each one

INBOX MANAGEMENT

Sort Your Inbox in Plain English

Describe how you want your inbox organized — Claude bulk-archives, labels, and sorts emails based on rules you explain in plain language

ADMIN AUTOMATION

Fill Any Form from One Prompt

Describe the record once — Claude fills CRM entries, admin forms, and repetitive application fields without you typing the same information over and over

POWER WORKFLOWS

Prompts as /Slash Commands

Save your best-performing prompts as slash commands — trigger a full multi-step workflow instantly without retyping a single instruction

THE SHIFT Hours of inbox backlog and repetitive admin work — handled in plain English, without touching a single filter or form

Automation & Integrations

Connect your tools and run workflows on autopilot — no API setup, no code

SCHEDULED AUTOMATION

Auto-Report Every Monday at 9 AM

Claude pulls metrics from your analytics, CRM, and financial tools on a schedule, compiles them into a standardized report, and saves it to Google Drive

DAILY BRIEFING

One Briefing Across All Platforms

Every morning, Claude scans Slack, Notion, and your team dashboard to surface priorities and connections you would have missed checking each tool separately

CONTENT FACTORY

3 Social Posts per Article, Auto-Made

Drop in a folder of articles — Claude reads each one and generates three ready-to-post captions, matched to your writing style and past performance data

CROSS-PLATFORM WORKFLOWS

Every Tool, One Workflow, Zero Code

Pull from Notion, Slack, Gmail, Calendar, and GitHub in a single workflow — Claude connects your entire stack and runs multi-step tasks without a line of code

THE POINT Set it once — Claude connects your stack and runs the whole chain, so you never have to be the glue between your tools

WHAT IS A SKILL?

Claude is already brilliant.

A Skill makes it yours.

CLAUDE BY DEFAULT

Brilliant, but generic.

Claude is highly intelligent — but it doesn't know your specific process, your standards, or your way of doing things.

- Your specific workflow or process
- Your quality standards
- Your tone and voice
- What 'done well' looks like to you

+

A SKILL

Your onboarding guide for Claude.

A plain-language file that teaches Claude exactly how to handle a task — your way, your standards, every time.

- Step-by-step task instructions
- Your quality criteria & format
- Your tone, voice, and style
- Domain expertise & context

=

THE RESULT

Your expert, on demand.

Claude now operates exactly like a specialist trained in your method — instantly, every session, for your whole team.

- Does it YOUR way, every time
- Zero re-briefing each session
- Consistent quality at scale
- Share it with your entire team

> **Think of a Skill like writing the perfect onboarding doc for a brilliant new hire.**
>
> You write it once. From that moment, Claude knows your process, your standards, and your way of doing things — perfectly, every single time.

Plain language	No code needed
Triggered via	/slash commands
Runs	every session

WHY SKILLS?

Skills beat every alternative.

Here's why.

ALTERNATIVE 01

Custom GPTs & AI Projects

Flexible, but isolated from everything else.

- Easy to create — no code needed
- Isolated — lives in a separate window
- Static — doesn't improve over time
- One chat window = one locked context

Convenient but limited

BEST CHOICE

Skills

Structured AND flexible. Lives inside your workflow.

- Plain language — anyone can write one
- Lives inside your existing Claude workflow
- Self-improving — update it any time
- One agent can run thousands of skills

Powerful. Flexible. No code.

ALTERNATIVE 02

N8N / Make.com & Automation

Structured, but rigid and technical.

- Consistent and repeatable once built
- Requires technical setup & maintenance
- Rigid — breaks when context changes
- No reasoning — purely deterministic

Powerful but developer-only

GPTs give you flexibility but no structure. Automation gives you structure but no intelligence. **Skills** give you both — and anyone on your team can write one.

No code. No technical setup. Just plain language — and Claude does the rest.

EXERCISE

Activate & Use a Skill

Pick the skill you like most · enable it · use it for real · share what you created

1 Enable a Skill

STEP 1 OF 3

- Open Claude and navigate to the Customize tab
- Browse the full list of Anthropic-provided skills
- Enable the one that excites you most — your choice, your style

2 Put It to Work

STEP 2 OF 3

- Use the skill exactly the way it is designed to be used
- Write a script · generate a document · create a PDF · build a visual · anything goes
- Explore it fully — let the skill do what it does best

3 Share in the Community

STEP 3 OF 3

- Go to our dedicated exercise post in the Skool community (link is in the video resources)
- Post a screenshot of your result, the file you created (PDF, JPG...), or a short text
- See what skills your fellow students are discovering — every result counts!

HOW TO SHARE YOUR RESULT

- Post a Screenshot
- Upload a File (PDF / JPG)
- Write a Short Description

EXERCISE 2

Build a Custom Skill

Use Claude's skill-creator to bring your own skill to life — choose your method and build it

1 Choose Your Method

STEP 1 OF 3

Choose one of the two methods we covered in this section:

METHOD 1 **Transform a Document**
Attach a doc — Claude turns it into a skill

METHOD 2 **Comprehensive analysis**
Fulfill a structured template to define your skill

2 Build with Skill-Creator

STEP 2 OF 3

- Open Claude and trigger the skill-creator skill
- Follow your chosen method — Claude will ask the right questions and build it for you
- Your skill is saved and ready to use immediately — test it right away

3 Share in the Community

STEP 3 OF 3

- Head to the dedicated exercise post in the Skool community (link in video resources)
- Share a screenshot of your new skill in action, the SKILL.md file, or describe what it does
- See what skills your classmates created — every skill is unique!

HOW TO SHARE YOUR RESULT

- Screenshot of Skill in Action
- Upload the SKILL.md File
- Describe What Your Skill Does

WHY SKILLS MATTER

Why Skills Matter

Three approaches to extending AI — only one truly adapts.

✗ Custom GPTs

Isolated and limited

- Isolated — hop between windows
- Don't self-improve over time
- Limited context & capabilities
- One project = one conversation

Limited and siloed

✗ Automation Tools

N8N, Make, Zapier

- Hardcoded, rigid workflows
- Deterministic only — no AI reasoning
- No human in the loop
- Breaks when anything changes

Rigid and fragile

✓ Skills

The adaptive approach

- Flexible — adapts to any situation
- Human in the loop when needed
- Self-improving with every use
- Thousands per agent, by prompting

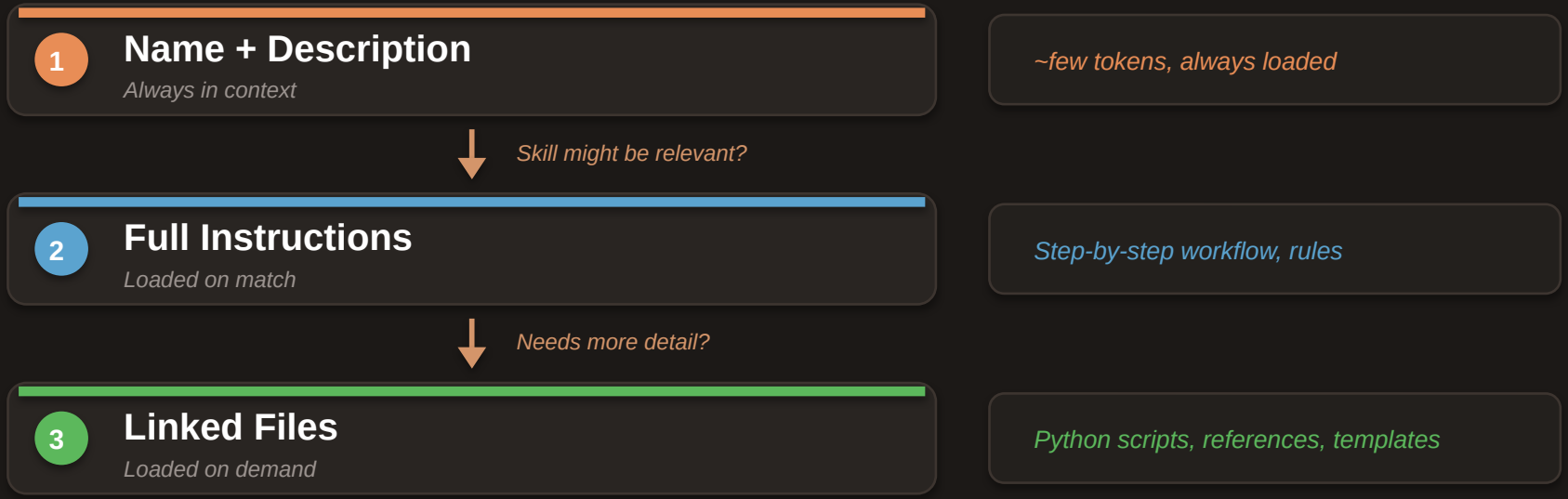
Flexible and powerful

Skills adapt, improve, and scale — the other approaches don't.

PROGRESSIVE DISCLOSURE

Progressive Disclosure — 3 Levels

Claude only loads what it needs — keeping context lean and efficient.



Only load what's needed — keep your context window lean and fast.

DESIGN PATTERNS

5 Design Patterns

Reusable structures for building powerful skills.

1. **Sequential Workflow**
   Steps in fixed order. Each depends on the previous.
   Account → Payment → Subscription → Welcome Email

2. **Multi-MCP Coordination**
   Orchestrate multiple services in phases.
   Figma → Drive → Linear → Slack

3. **Iterative Refinement**
   Generate → Review → Improve → Repeat.
   Marketing images, writing, code review

4. **Context-Aware Branching**
   Same input, different paths based on type.
   Code → GitHub MCP / Doc → Notion MCP

5. **Domain Intelligence**
   Company knowledge embedded in a skill.
   SOPs, deployment rules, microservice guides

Pattern → Skill → Reuse — build once, apply everywhere.

SKILLS + MCPs

The Kitchen Analogy

Two halves of the same system — tools and instructions.

MCPs = The Hands

Connect to services, provide tools, access real-time data.

- Execute actions in external systems
- Read and write real-time data
- Always loaded once configured

Provides capability

Skills = The Recipes

What ingredients, what order, what amounts, what output.

- Step-by-step instructions for Claude
- Domain knowledge and best practices
- Only loaded when relevant

Provides intelligence

MCPs provide the tools — Skills provide the instructions on how to use them.

SKILL CATEGORIES

3 Categories of Skills

Every skill falls into one of these three buckets.

1 Document & Asset Creation

Create polished outputs

- PDFs and presentations
- Reports and spreadsheets
- Images and design assets
- Professional documents

Output-focused

2 Workflow Automation

Repeatable processes

- Multi-step task sequences
- Processes you repeat often
- Cross-platform orchestration
- Team-wide standard workflows

Process-focused

3 MCP Enhancement

Guide Claude on MCP usage

- Teach Claude how to use an MCP
- API-specific best practices
- Error handling strategies
- Optimal tool combinations

Most underutilised

Intelligence-focused

Documents, Workflows, MCP Enhancement — pick the right category for your skill.

YAML FRONT MATTER

YAML Front Matter

The only part Claude always keeps in memory.

```
---
name: csv-data-pipeline
description: Cleans, validates and
  transforms CSV files...
---
```

name
kebab-case, 1–4 words, descriptive

description
What it does + When to use it
+ Trigger words

This is the ONLY part Claude always keeps in memory

- **What it does**
  Core function of the skill
- **When to use it**
  Context that triggers loading
- **Trigger words**
  Keywords Claude matches on

Name + Description = your skill's identity — get these right first.

BAD VS GOOD

Bad vs Good Descriptions

Pair 1 — Vague vs Specific

Bad

"helps with projects"

Why it fails:

Too vague. Every skill could match this. Claude has no way to decide when to load it.

Good

Analyzes Figma design files and generates developer handoff documents. Use when user uploads .fig files or requests design specs.

Why it works:

Specific action + file type triggers + clear use case. Claude knows exactly when this skill applies.

Specific beats vague — tell Claude exactly what the skill does and when.

BAD VS GOOD

Bad vs Good Descriptions

Pair 2 — Impressive but useless vs Actionable

Bad

"Creates sophisticated multi-page documentation systems"

Why it fails:

No triggers. Sounds impressive but tells Claude nothing about when to activate.

Good

Manages Linear project workflows including sprint planning, task creation, and status tracking. Use when user mentions sprint, linear tasks, or create tickets.

Why it works:

Names the service + lists actions + provides trigger phrases Claude can match on.

Trigger words matter — Claude matches on keywords, not impressiveness.

BAD VS GOOD

Bad vs Good Descriptions

Pair 3 — Buzzword salad vs Real-world clarity

Bad

"Implements the project entity model with hierarchical relationships"

Why it fails:

Buzzword salad. Too technical. No user would type these words. Zero trigger value.

Good

End-to-end customer onboarding for PayFlow. Use when user asks to set up a new customer, configure billing, or create subscription plans.

Why it works:

Domain-specific + real actions users actually ask for + product name as trigger.

Write for humans — use the words your users would actually type.

THE GOLDEN RULE

The Golden Rule for Descriptions

A simple formula that always works.

What it does
Core function of the skill

+

When to use it
Context + file types + user intent

+

Key trigger phrases
Words users actually type

= Perfect Description

- **Keep under 1,000 characters**
- **Triggers can be text or event-based**

What + When + Triggers — the formula for every great skill description.

INSTRUCTIONS

Bad vs Good Instructions

Structure beats vagueness — every time.

X Bad Instructions

"Help the user with their data. Validate it and make sure everything looks good. Process the data appropriately and handle any issues."

Vague — different results every time

✓ Good Instructions

- **Step 1: Inspect**
  Check rows, columns, types, missing values
- **Step 2: Identify**
  Decision framework for each issue type
- **Step 3: Apply fixes**
  Log every change made to the data
- **Step 4: Export**
  Descriptive filename + summary report

Structured — consistent results every time

Steps > paragraphs — structure your instructions for consistent output.

SKILL CHECKLIST

What to Include in Your Skill

Seven essentials for every well-built skill.

- ✓ **Description with trigger words**
- ✓ **Structured step-by-step instructions**
- ✓ **Clarifying questions for missing info**
- ✓ **Output format specification**
- ✓ **Multiple variations (let user choose)**
- ✓ **Rules section (what should never happen)**
- ✓ **Reference files for examples and context**

Make reference file usage obligatory, not optional

7 essentials — check every box before shipping your skill.

TEST METHODS

3 Test Methods

Trigger, function, and value.

1

Triggering Test

Does the skill activate when it should?

Use a NEW session

2

Functional Test

Is the output correct and consistent?

Run 4–5 times

3

Value Test

Is this skill actually worth having?

Complexity vs benefit

Three tests — does it trigger, does it work, is it worth it?

TRIGGERING

The Goldilocks Principle

Finding the sweet spot for skill triggering.

Goldilocks — test with prompts that should AND should not trigger.

TEST PROMPTS

Testing Prompts Example

For a CSV cleaning skill.

✓ **Should trigger:**

“I’ve got this messy CSV file, can you clean it up?”

X **Should NOT trigger:**

“Can you help me write a Python script?”

? **Grey area:**

“I need to work with some data”

Test all three zones — yes, no, and maybe prompts.

TEST TIPS

Functional & Value Test Tips

What to look for in each test.

2 Functional Test Tips

- Run 4–5 times with different inputs
- Inconsistent output? → Tighten instructions
- Add output examples to references
- Test with sub agents — behaviour may change

3 Value Test Tips

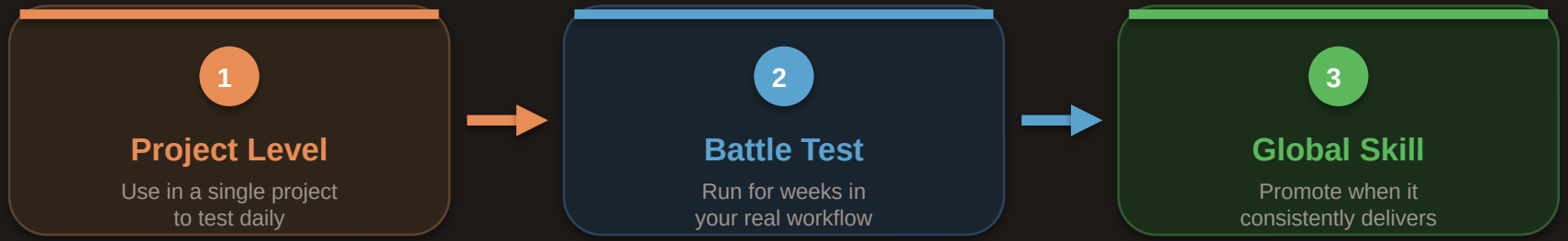
- Is it saving time and improving quality?
- Not everything needs to be a skill
- Maybe a simple Python script is better
- Be honest — scrap it if it's not worth it

Iteration: test → fix → test again — ask Claude Code to help update the skill.

GO GLOBAL

Battle Testing Before Going Global

The path from project-level to global skill.



- Don’t make a skill global until you’ve used it in your real workflow for at least a few weeks and it consistently delivers good results.

Project → Battle test → Global — earn the promotion.

BAD STRUCTURE

Bad Folder Structure

Everything crammed into one file.

Bad

```
my-skill/
└── skill.md   ← all in one file
```

Problems:

No reference files
No reusable scripts
Hard to maintain or update

No references, no scripts, hard to maintain

One file = one problem — you'll outgrow it fast.

GOOD STRUCTURE

Good Folder Structure

Clean separation. Claude loads files only when needed.

✓ Good

```
csv-data-pipeline/
├── skill.md          ← main instructions
├── references/
│   └── column-types.md   ← handling rules
└── scripts/
    └── clean.py      ← data processing
```

- **skill.md** — Always loaded first
- **references/** — Loaded on demand
- **scripts/** — Executed when needed

Clean separation. Claude loads files only when needed.

Organised folders = maintainable skills — split by purpose.

3 RULES

3 Rules for Folder Structure

Keep it simple, keep it clean.

1

Descriptive file names in lowercase with dashes

`clean-csv-data.py` *not* `script1.py`

`.py`

2

Don’t over-nest — one level of folders is enough

If you need sub-sub-folders, split into two skills

1

3

Keep reference files focused — one topic per file

Claude pulls in only what it needs

1:1

Simple, descriptive, focused — three rules for clean skill folders.

DIAGNOSE & FIX

What to Update When Things Go Wrong

Match the symptom to the fix.

Process is wrong (skips steps, wrong order)	→	**Update skill.md instructions**
Needs more info (brand, audience, context)	→	**Add a reference file**
Does something it shouldn't	→	**Add a rule to rules section**
Struggles with MCP/tool	→	**Guide manually, then create MCP reference doc**

Self-improving skills

- Save approved outputs as good examples automatically
- Add user corrections to rules section automatically
- The more you use it, the better it gets

Symptom → Fix — match the problem to the right file.

COMMON SCENARIOS

Common Editing Scenarios

Three patterns you'll see again and again.

Too Rigid

Output format too strict, can't adapt

Fix:

Loosen instructions, allow user preferences

Flexibility

Too Loose

Different results every time

Fix:

Add more steps, examples, clearer output format

Consistency

Too Big

Skill does too many things

Fix:

Split into two focused skills

Focus

Rigid, loose, or big? — diagnose the pattern, apply the fix.

SUMMARY

Summary

Three ways to edit your skills.

1

Manual in VS Code

Full control, edit files directly

2

Ask Claude Code

Describe what you want, Claude edits

3

Claude Cowork

Chat about changes, conversational

- Always test after each change. The best skills evolve with your workflow.

Edit → Test → Improve — skills get better with every iteration.

What is a Plugin?

Everything Claude needs for a specific workflow — bundled, installed, and ready to go

DEFINITION

A plugin is a ready-made bundle of capabilities you install into Claude. Once installed, it immediately gives Claude new skills, direct connections to external services, and custom configuration — turning Claude into a purpose-built tool for your workflow.

Like an app you install
for Claude

WHAT IT ADDS

Extends Claude's Capabilities

Gives Claude skills and knowledge it doesn't have by default — built specifically for your industry, team, or task type

WHAT IT CONNECTS

Integrates With Your Apps

Links Claude directly to the tools you already use — Notion, Slack, GitHub, Google Drive — so it can read and act in real time

WHAT IT SHAPES

Configures Claude's Behavior

Tells Claude how to think, what to remember, and how to respond — so it behaves like a trained teammate, not a generic assistant

WHAT'S INSIDE

Skills + Connectors + Commands = Plugin

UNDERSTANDING THE DIFFERENCE

Skills vs Plugins

SKILLS

⚡ Workflow Instructions

Step-by-step guidance that tells Claude how to tackle a specific task — written in plain language.

- Lives inside a SKILL.md file
- Gives Claude domain expertise & best practices
- Triggered by slash command or natural language
- Self-contained — no external connections required
- Can be created without any code

VS

PLUGINS

Installable Capability Bundle

A packaged unit that bundles Skills, MCP connectors, and tools — installed in one click.

- Contains Skills + MCP servers + Tools + Config
- Connects Claude to live external apps & data
- Installed from a marketplace in one click
- Scoped permissions — you control what it can do
- Versioned, shareable, and team-deployable

BETTER TOGETHER

A Skill is the brain — it teaches Claude exactly how to do something. A Plugin is the body — it delivers that skill, connects it to live data, and makes it installable for your whole team.

Analogy: The Plugin is the app you install. The Skill is the feature built inside it.

PLUGIN ARCHITECTURE

Every plugin is built from 3 core components.

01 Skills

The intelligence layer

Expert workflow instructions that teach Claude exactly how to complete a task — step by step, every time.

- Written as plain-language SKILL.md files
- Encode domain expertise & best practices
- Triggered by slash commands or prompts
- Reusable across sessions and users

Example: /pptx, /research, /docx

02 Connectors

The reach layer

MCP servers that give Claude live, two-way access to external apps and data sources in real time.

- Built on the open Model Context Protocol
- Connect to Slack, Notion, Jira, Drive & more
- Read and write to external systems
- Scoped — only the access you grant

Example: Slack MCP, GitHub MCP, Notion MCP

03 Commands

The control layer

Automatable shortcuts that let users invoke specific actions on demand — or have them run on a schedule.

- Named shortcuts with a single trigger
- Can be run manually or on a schedule
- Chain multiple skills & connectors together
- Shareable across an entire team or org

Example: /weekly-report, /sync-jira

Skills + Connectors + Commands = Plugin

EXERCISE

Install & Use a Plugin

Pick a plugin · install it · put it to work · share what you discovered

1 Install a Plugin

STEP 1 OF 3

- Open Claude and go to the Plugins tab in the Customize menu
- Browse the Anthropic-provided plugins — pick the one that fits your workflow best
- Install it in one click — it activates immediately, no setup required

2 Put It to Work

STEP 2 OF 3

- Use the plugin the way it is designed — trigger a skill, run a command, or connect to an app
- Let it do something real: pull data, generate a document, automate a task, anything goes
- Explore it fully — see exactly what the plugin adds to your Claude experience

3 Share in the Community

STEP 3 OF 3

- Go to our dedicated exercise post in the Skool community (link is in the video resources)
- Post a screenshot of your result, the file you created (PDF, JPG...), or a short text
- See what plugins your fellow students are discovering — every result counts!

HOW TO SHARE YOUR RESULT

- Post a Screenshot
- Upload a File (PDF / JPG)
- Describe What You Did

CLAUDE CODE

What is Claude Code?

The developer version of Claude — works directly in your terminal and codebase, not in a chat panel.

DEFINITION

Claude Code is a command-line tool that brings Claude directly into your terminal and codebase. Instead of chatting in a browser, you delegate coding tasks — and Claude reads, edits, runs, and fixes your actual files.

HOW IT DIFFERS FROM COWORK

Cowork:	Visual desktop tool — no code needed
Code:	Command-line tool — for developers only
Cowork:	Works with your files and apps
Code:	Works inside your code repository

WHAT CLAUDE CODE CAN DO — FOR DEVELOPERS

CAPABILITY 01

Write & Edit Code

- Writes new functions, classes, and modules from scratch
- Edits existing code across multiple files in one go
- Refactors, renames, and reorganizes entire codebases
- Understands your project structure before making changes

CAPABILITY 02

Run & Debug

- Executes scripts and reads the real output directly
- Spots errors in stack traces and fixes them automatically
- Re-runs code after each fix until tests pass
- No copy-pasting errors into a chat — it sees them live

CAPABILITY 03

Navigate Codebases

- Reads and understands your full project structure
- Finds relevant files without being told where to look
- Traces function calls and dependencies across files
- Explains what existing code does in plain English

CAPABILITY 04

Git & Terminal

- Stages, commits, and pushes code changes via git
- Creates branches, manages pull requests
- Runs shell commands, installs packages, sets up projects
- Works entirely inside your existing dev environment

THE DISTINCTION

Claude Code is for developers living in the terminal. It closes the loop between writing and running code. **Cowork** closes the loop between instructions and real-world file tasks. *Same Claude — different environment, different user.*

WHAT IS AN IDE?

Three tools. One app.

Integrated Development Environment

01 File Explorer

See your whole project

Like Finder on Mac or File Explorer on Windows — built right into the app.

- **Browse folders & files visually**
- **Click to open any file instantly**
- **See your entire project structure**

Think: Finder / File Explorer

02 Text Editor

Read and write code

Like Notepad or Notes — but purpose-built for code with color highlighting.

- **Syntax highlighting for readability**
- **Navigate large files easily**
- **Designed specifically for code**

Think: Notepad, but smarter

03 AI Chat Window

Your built-in assistant

Like claude.ai or ChatGPT — but built directly into where your code lives.

- **Ask questions without leaving**
- **AI sees your code in context**
- **Get help right where you work**

Think: Claude, built in

File Explorer + Text Editor + AI Chat = **IDE**

Everything in one place — so you never have to jump between apps again.

GIT VS GITHUB

The tool vs. the storage.

Two different things that work together.

Git

The tool on your computer

- **Tracks every change you make**
- **Creates snapshots of your project**
- **Unlimited undo for your code**

Think: version history in Google Docs

GitHub

The website in the cloud

- **Stores your code online safely**
- **Backup if your laptop breaks**
- **Access from anywhere, any device**

Think: Google Drive for your code

Git tracks your changes + **GitHub** saves them online = **Full backup, always**

Claude Code handles all of this automatically — you don't need to learn any Git commands.

THE #1 RULE

Garbage in, garbage out.

Better input = better output. Here's the simple framework.

01 Think in Features

Not products — features

Don't say "build me an app." Break it into specific features — each one becomes a clear task.

- **List every feature individually**
- **Each feature = one clear task**
- **Features together = your product**

Product = list of features

02 Be Specific

Describe exactly what you want

Like briefing a talented new hire — the more detail you give, the better the result.

- **What it does, who it's for**
- **How it should look & behave**
- **Say it clearly, get it right**

Clear input = clear output

03 Test As You Go

Catch problems early

After each feature, ask Claude to test it. Don't build on broken foundations.

- **Test each feature before next**
- **Fix problems while they're small**
- **Never stack untested code**

Build → Test → Repeat

Think in features + Be specific + Test as you go = **Dramatically better results**

The model is powerful enough. If you're getting bad results, it's almost always the input.

HOW CLAUDE CODE WORKS

What are Sub-Agents?

Let the main agent manage — let sub-agents do the work.

Main Agent

The Manager

- Coordinates all tasks
- Stays clean and focused
- Collects results and summaries

Explorer Agent

Searches codebase and finds files

Coder Agent

Writes and edits code

Reviewer Agent

Reviews code for quality and bugs

Returns short summary only

Each sub-agent gets its own context window — so none of their work fills up the main conversation.

CONTEXT WINDOW MANAGEMENT

Why Sub Agents Matter

Protect your main conversation — keep quality high throughout the build.

Without Sub Agents

Context fills up fast

- All work happens in one conversation
- Context hits 80% and gets compacted
- Important details get lost — quality drops

Main conversation overloaded

the fix

With Sub Agents

Main stays clean

- Each agent gets its own context window
- Only short summaries return to main
- Main stays at 15% — quality stays high

Main conversation protected

Each sub agent gets its own context window — none of their work fills up the main conversation.

Sonnet, Opus & Haiku

Speed, cost & intelligence tradeoffs — how to pick the right model for every task

SPEED FIRST

Haiku

Fast & Lightweight

PERFORMANCE

SPEED

COST

INTELLIGENCE

BEST FOR

- Quick answers and simple Q&A
- High-volume automated tasks
- Real-time responses & chat

When speed matters most

✦ RECOMMENDED

Sonnet

Balanced & Capable

PERFORMANCE

SPEED

COST

INTELLIGENCE

BEST FOR

- Writing, analysis & research
- Code, documents, daily work
- Your default — 90% of tasks

Your go-to for almost everything

MOST CAPABLE

Opus

Deep & Powerful

PERFORMANCE

SPEED

COST

INTELLIGENCE

BEST FOR

- Complex multi-step reasoning
- Nuanced strategy & deep analysis
- When only the best will do

When max intelligence is needed

QUICK PICK

90% of the time → **Sonnet** Speed-critical → **Haiku** Hardest problems → **Opus**

When in doubt, start with — switch only when you have a specific reason to

How Claude Reads Documents & Images

What actually happens when you upload a file or share an image — and what to expect

PDF / DOCUMENTS

How Claude reads your files

1. Upload triggers text extraction — the system pulls all text content from the PDF as plain text
2. That text is tokenized and loaded into Claude's context window — like sending a very long message
3. Claude reads it exactly like your chat messages — to Claude, it's all just text

IMPORTANT LIMITATIONS

- Scanned PDFs are image files — Claude sees a picture, not extractable text
- Very large PDFs may be truncated if they exceed the context window limit
- Tables and complex formatting can lose their structure during extraction

IMAGES

How Claude "sees" what you share

1. A visual encoder processes the image — pixels are converted into numerical vector data
2. That data gets translated into representations Claude's language model can understand
3. Claude 'reads' those vectors the same way it reads any other text in context

GOOD AT

- Text in images
- Objects, people & layouts
- Charts & graphs
- UI screenshots

STRUGGLES WITH

- Very small text
- Handwriting (sometimes)
- Complex image tables
- Exact pixel details

KEY INSIGHT Claude doesn't "open" files the way your computer does. Everything — documents, images, and your messages — gets converted into the same token stream before Claude reads it.

EXERCISE

Build Your Own Claude Project

Create a Project, add your files and instructions — then watch Claude respond like a specialist every time

1 Create Your Project

STEP 1 OF 3

- Go to Claude, create a new Project, and give it a clear name and purpose
- Write a system prompt: describe how Claude should behave in this project
- Bonus: make it specific — 'my Notion assistant' beats 'general helper'

2 Add Files & Instructions

STEP 2 OF 3

- Upload files Claude should always have: SOPs, brand docs, or examples
- Add background in the instructions — who you are and what you need from Claude
- Test it: ask something that only works if Claude actually read your files

3 Test It and Share

STEP 3 OF 3

- Screenshot your setup — project name, instructions, and the files you added
- Share in Skool: what's the project for and how does it change Claude's answers?
- Bonus: show a before/after — same question in project vs. a regular chat

HOW TO SHARE YOUR RESULT

- Show Your Setup
- Share What It Does
- Demo the Difference

Memory — How It Works & How to Use It

What Claude remembers, what it forgets, how to control it, and how to use memory edits.

WITHIN SESSION

What It Remembers

- Everything said in the current conversation
- Files and docs shared in this session
- Custom instructions — always active
- Project context if working inside a Project

ACROSS SESSIONS

What It Forgets

- Conversation history once the chat ends
- Personal details, preferences, past decisions
- Files shared — unless re-uploaded or in a Project
- Anything not saved to memory or Project context

PERSISTENCE TOOLS

How to Control It

- Projects: persistent context across all sessions
- Custom Instructions: always-on preferences and style
- Memory feature: Claude stores key facts about you
- New conversation = clean slate (use to reset context)

MANAGE YOUR MEMORY

Memory Edits

- View everything Claude has stored about you
- Edit or delete individual memory entries
- Add facts: tell Claude to remember something
- Memory is transparent — you are always in control

THE KEY INSIGHT

By default, Claude's memory resets every conversation. **Projects and memory settings** are how you make it persistent — so Claude actually knows your context, preferences, and history over time.

Prompt Engineering for Real Results

Not theory — practical patterns: role + task + format, chain of thought, iterative prompting. With live examples.

THE CORE FORMULA

ROLE *You are a...* + **TASK** *Do this...* + **CONTEXT** *Given that...* + **FORMAT** *Output as...* = **RESULT** *Great output*

PATTERN 01

Role + Task + Format

Gives Claude a clear persona, objective, and output structure

EXAMPLE PROMPT

You are a product manager. Write a 3-bullet summary of this user feedback. Use plain English, no jargon.

WHEN: Use this as your default starting point for any new task

PATTERN 02

Chain of Thought

Asks Claude to reason step by step before giving a final answer

EXAMPLE PROMPT

Walk me through your reasoning step by step, then give me your final recommendation at the end.

WHEN: Use for complex decisions, analysis, or anything that needs logic

PATTERN 03

Iterative Prompting

Start broad, then refine with follow-up messages to improve the output

EXAMPLE FLOW

1. Get a draft output 2. "Make it shorter" 3. "Now make the tone more direct" 4. "Add a CTA at the end"

WHEN: Use when you're not sure exactly what you want yet — explore then refine

REMEMBER

You don't need to memorize prompting patterns. **Start with Role + Task + Format** for any new task, then use **follow-up messages to refine**. The more specific your input, the more useful Claude's output.